

Rapport de veille technologique frontend

Baptiste Appriou

01 Vue

Vue est un framework frontend qui permet de construire des interfaces avec JavaScript. Dans ce projet, j'utilise Vue 3 avec TypeScript. Un fichier .vue regroupe la logique du composant, son HTML et ses styles. Cette organisation me permet de retrouver plus facilement ce qui gère l'affichage et les actions.

Pour le Kanban, le découpage en composants aide à réutiliser le code. Les pages de connexion et d'inscription partagent déjà le même formulaire. Avec v-model, les champs sont reliés aux données du script. Les ref permettent de mettre à jour l'affichage, notamment quand un message d'erreur apparaît.

Vue Router gère les pages, tandis que Pinia centralise l'état de la connexion. Ce sont des outils complémentaires à Vue. Le token est conservé dans localStorage pour rester connecté après un rechargement. Les appels au back passent par des fichiers séparés, ce qui rend leur rôle plus clair.

En contrepartie, il faut comprendre la réactivité et savoir où placer les données. Une valeur dans le composant, dans Pinia ou dans localStorage ne sert pas au même besoin. Vue me semble adapté pour apprendre progressivement, mais il demande tout de même de bonnes bases en JavaScript.

02 Angular

Angular est un framework frontend qui s'appuie sur TypeScript. Il organise l'interface en composants, avec un template HTML, des styles et une classe qui gère les actions. Cette séparation donne des repères pour retrouver ce qui concerne l'affichage et ce qui concerne les traitements. Les composants peuvent ensuite être réutilisés dans plusieurs pages.

Pour le Kanban, Angular propose des outils pour gérer les formulaires. On peut contrôler les champs obligatoires, vérifier une adresse email et afficher les erreurs de saisie. Ces outils répondent donc à des besoins présents dans mon projet, notamment pour la connexion et l'inscription des utilisateurs.

Son avantage me semble être le cadre commun qu'il donne à l'équipe. Les développeurs peuvent suivre les mêmes conventions et s'appuyer sur les outils du framework. Le TypeScript est aussi un point de repère après mon projet backend avec NestJS, même si le fonctionnement du front reste différent.

En contrepartie, il faut prendre le temps de comprendre ses conventions et les outils proposés. Pour mon apprentissage, cela représente davantage de notions à découvrir ensemble. Je compare ici Angular à partir de sa documentation.

03 React

React est une bibliothèque JavaScript qui sert à construire des interfaces à partir de composants. Chaque composant possède sa logique et décrit ce qu'il affiche. Le JSX permet d'écrire une structure proche du HTML dans le code JavaScript. Cela donne une autre façon de regrouper l'affichage et les actions d'un formulaire.

Pour le Kanban, cette approche permettrait de partager un formulaire entre la connexion et l'inscription, comme dans mon projet Vue. L'état du composant sert à gérer les valeurs saisies et les messages affichés. Quand cet état change, React met à jour l'interface. Les données peuvent également être transmises entre les composants.

Son avantage est la liberté laissée dans l'organisation de l'application. Pour avoir plusieurs pages et des données partagées, il faut aussi décider comment gérer la navigation et l'état. Les choix se font autour de React, ce qui permet d'adapter les outils au projet, mais ajoute des décisions pour un débutant.

La difficulté serait donc, pour moi, d'apprendre à la fois la syntaxe JSX et l'organisation des outils retenus. Je compare React à partir de sa documentation, sans avoir réalisé ce Kanban avec cette bibliothèque.

04 Pourquoi je choisis Vue

J'ai choisi Vue pour construire le front de mon Kanban tout en restant dans l'écosystème JavaScript et TypeScript utilisé avec NestJS. Je cherche surtout un compromis entre la prise en main, l'organisation du code et les besoins du projet. Le découpage d'un fichier .vue me donne des repères pour comprendre ce que j'écris.

Le dépôt illustre ce choix : les pages de connexion et d'inscription utilisent le même composant AuthForm. Les champs sont reliés aux données avec v-model et les erreurs apparaissent grâce aux valeurs réactives. Pinia centralise la connexion, tandis que Vue Router protège l'accès au board. Les requêtes passent par un client basé sur fetch.

Je garde toutefois plusieurs limites en tête. Le store ne conserve pas sa valeur après un F5 : le token doit être enregistré dans localStorage, puis restauré. Le back reste responsable de vérifier ce token. Le Kanban est encore en cours de construction ; je m'appuie donc surtout sur l'authentification déjà réalisée.

Enfin, dans un cadre plus personnel, j'ai eu envie de choisir Vue car je voulais aller vite avec un framework qui me semblait plus simple à apprendre de zéro. Cependant, dans le cadre de mon alternance j'aurais plus tendance à partir sur Angular afin d'avoir une structure plus complète bien que plus lente.

Documentations officielles

[Vue : introduction](#) ; [Vue : formulaires](#)

[Pinia : introduction](#)

[Angular : composants](#) ; [Angular : formulaires](#)

[React : démarrage](#) ; [React : création d'une application](#)